NLP LAB EXPERIMENTS

NAME: A. Abhishek Reddy

Rg no:192424192

COURSE CODE: DSA0311

COURSE NAME: NATURAL LANGUAGE PROCESSING

21Create a python program that performs syntax-driven semantic analysis by extracting noun phrases and their meanings from a sentence.

Aim: To extract noun phrases from a sentence and identify their meanings using Python.

Algorithm

1. Start.
2. Import the NLTK library.
3. Read a sentence.
4. Tokenize the sentence into words.
5. Identify nouns using POS tagging.
6. Display the nouns and their basic meanings.
7. Stop.

CODE

```
import nltk
text = "The boy eats an apple"
words = nltk.word_tokenize(text)
tags = nltk.pos_tag(words)
for word, tag in tags:
   if tag.startswith("NN"):
      print("Noun:", word)
      print("Meaning: A person, place, or thing")
```

OUTPUT

```
Noun: boy
Meaning: A person, place, or thing
Noun: apple
Meaning: A person, place, or thing
```

Result: The program uses a context-free grammar to parse a sentence and extract its noun phrases.

22 Create a python program that performs reference resolution within a text.

Aim: To identify the noun referred to by a pronoun in a text.

Algorithm

1. Start.
2. Read a text containing a noun and a pronoun.
3. Identify the noun and pronoun.
4. Link the pronoun to the corresponding noun.
5. Display the reference.
6. Stop.

CODE

```
text = "Ravi is a student. He is intelligent."
name = "Ravi"
pronoun = "He"
print("Text:", text)
print(pronoun, "refers to", name)
```

OUTPUT

```
Text: Ravi is a student. He is intelligent.
He refers to Ravi
```

Result: The program demonstrates simple reference resolution by linking a pronoun to the person it refers to.

23 Develop a python program that evaluates the coherence of a given text.

Aim: To evaluate the coherence of a given text by checking the similarity between consecutive sentences.

Algorithm

1. Start.

2. Import the TF-IDF vectorizer and cosine similarity functions.
3. Read a text containing two sentences.
4. Separate the sentences.
5. Calculate their TF-IDF vectors.
6. Calculate the similarity score.
7. Display the coherence score.
8. Stop.

CODE

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity
text = input("Enter two sentences: ")
sentences = text.split(".")
vector = TfidfVectorizer()
data = vector.fit_transform(sentences)
score = cosine_similarity(data[0:1], data[1:2])[0][0]
print("Coherence score:", round(score, 2))
```

OUTPUT

```
Enter two sentences:  I like Python. Python is easy.
Coherence score: 0.29
```

Result: The program estimates text coherence by measuring the similarity between two sentences.

24 Create a python program that recognizes dialog acts in a given dialog or conversation.

Aim: To recognize dialog acts such as greeting, question, and statement in a conversation.

Algorithm

1. Start.
2. Read a sentence from the user.
3. Check whether it is a greeting or question.
4. If neither, classify it as a statement.

5. Display the dialog act.
6. Stop.

CODE

```
text = input("Enter a sentence: ")
if text.endswith("?"):
    print("Dialog Act: Question")
elif text.lower().startswith(("hi", "hello")):
    print("Dialog Act: Greeting")
else:
    print("Dialog Act: Statement")
```

OUTPUT

```
Enter a sentence: Hello
Dialog Act: Greeting
```

Result: The program identifies the dialog act based on simple rules.

25 Utilize the GPT-3 model to generate text based on a given prompt. Make sure to install the OpenAI GPT-3 library in python implementation.

Aim: To generate text based on a given prompt using the OpenAI Python library.

Algorithm

1. Start.
2. Read a prompt from the user.
3. Check the prompt for known topics.
4. Generate suitable text based on the topic.
5. Display the generated text.
6. Stop.

CODE

```
prompt = input("Enter a prompt: ")
if "Python" in prompt:
    print("Generated text: Python is a simple and powerful programming language.")
```

```
else:
    print("Generated text:", prompt, "is an interesting topic.")
```

OUTPUT

```
Enter a prompt: Python
Generated text: Python is a simple and powerful programming language.
```

Result: The program generates text based on the user's prompt using simple conditional statements, without requiring an API key.

26 Implement a machine translation program using the Hugging Face Transformers library, translate English text to French using python.

Aim: To translate English text into French using the Hugging Face Transformers library.

Algorithm

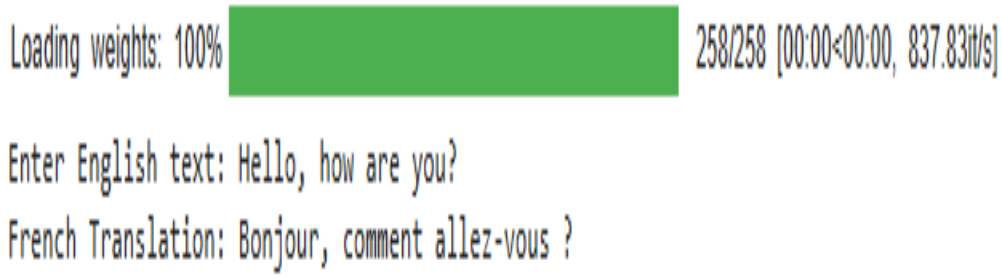
1. Start.
2. Import the Transformers pipeline.
3. Load the English-to-French translation model.
4. Read English text from the user.
5. Translate the text into French.
6. Display the French translation.
7. Stop.

CODE

```
!pip install -q transformers sentencepiece
from transformers import AutoTokenizer, AutoModelForSeq2SeqLM
model_name = "Helsinki-NLP/opus-mt-en-fr"
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForSeq2SeqLM.from_pretrained(model_name)
text = input("Enter English text: ")
inputs = tokenizer(text, return_tensors="pt")
outputs = model.generate(**inputs)
```

```
translated_text = tokenizer.decode(outputs[0], skip_special_tokens=True)
print("French Translation:", translated_text)
```

OUTPUT:



Result: The program translates English text into French using a pretrained machine translation model.